

# Types of Cross Validation

*Complete Notes with Detailed Step-by-Step Explanation*  
(Har chhoti se chhoti concept ko asaan bhasha mein samjhaya gaya hai)

## 1. Introduction — Yeh Topic Kya Hai?

Jab hum koi Machine Learning model banate hain, toh sabse bada sawaal yeh hota hai: "**Mera model kitna achha hai? Naye data par yeh kitna sahi kaam karega?**" Is sawaal ka sahi jawab dene ke liye hum apne data ko alag-alag hisson (parts) mein baantte hain.

**Cross Validation** ek aisi technique hai jisme hum apne training data ko baar-baar alag-alag tareeke se **Train** aur **Validation** parts mein todte hain, har baar model ko train karke check karte hain, aur fir un sabhi results ka **average** lekar model ki asli performance nikalte hain.

Is note mein teacher ne Cross Validation ke **5 types** samjhaye hain:

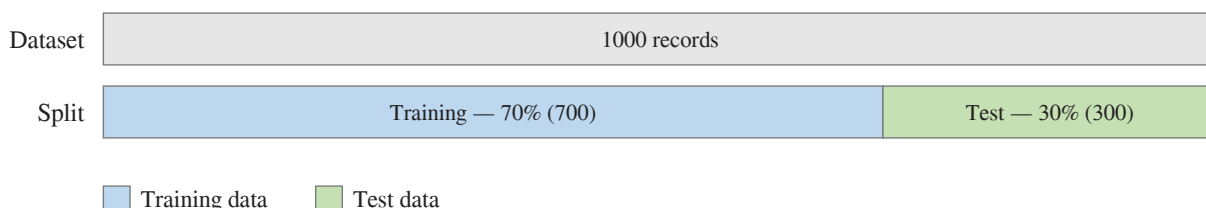
- **Leave One Out CV (LOOCV)**
- **Leave P Out CV (LPOCV)**
- **K-Fold CV**
- **Stratified K-Fold CV**
- **Time Series CV**

Lekin in types ko samajhne se pehle, pehle yeh samajhna zaroori hai ki Cross Validation ki *zaroorat* aakhir padti kyun hai. Chalo shuru se dekhte hain.

## 2. Dataset Ko Split Karna — Training aur Test

Teacher ne shuruaat ek **Dataset** se ki hai jisme **1000 records** (rows) hain. Is dataset ko pehle do hisson mein baanta jaata hai:

- **70% Training data** (= 700 records) — iska use model ko *sikhane* ke liye hota hai.
- **30% Test data** (= 300 records) — iska use model ki *final performance check* karne ke liye hota hai. Teacher ne likha hai: "*To check the performance of Model*".



Isko aise samjho: **Training data = padhai ke chapters** (jisse tum seekhte ho) aur **Test data = final board exam** (jisme tumhe woh sawaal milte hain jo tumne kabhi nahi dekhe). Agar exam ke sawaal pehle hi dekh liye, toh marks se tumhari asli understanding pata nahi chalegi — isliye Test data ko model ko training ke dauraan kabhi dikhaya nahi jaata.

### 3. Training Data Ke Do Kaam

Teacher ne diagram mein Training box ke saath do points likhe hain — training data ke **do main use** hain:

- ① **Training our Model** — model ko data se pattern seekhne dena.
- ② **Hyperparameter Tuning** — model ki settings ko best banana.

#### Hyperparameter Tuning Kya Hai?

Yahan do alag cheezein samajhni zaroori hain:

- **Parameters** — jo model *khud seekhta hai* training ke dauraan. Jaise Linear Regression mein  $\theta_0$  (intercept) aur  $\theta_1$  (slope).
- **Hyperparameters** — jo *hum khud pehle se set karte hain*, model unhe seekhta nahi. Jaise KNN mein **K** ki value, Decision Tree ki **max\_depth**, ya Gradient Descent ka **learning rate ( $\alpha$ )**.

**Hyperparameter Tuning** ka matlab hai — alag-alag hyperparameter values try karke dekhna ki kaunsi value par model sabse achha perform karta hai. Jaise K = 3, 5, 7, 9 try karo aur jis par accuracy sabse zyada aaye, wahi K choose karo.

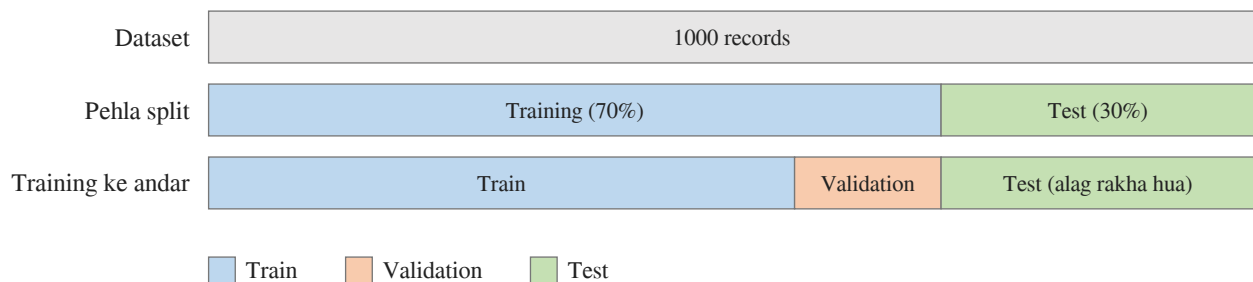
#### Ab Problem Kahan Hai?

Tuning karne ke liye har hyperparameter value ki performance check karni padegi. Par yeh check **Test data par nahi kar sakte** — kyunki agar hum baar-baar Test data par check karke apne hyperparameters chunenge, toh Test data ka bhi indirectly model ko pata chal jaayega, aur woh 'final exam' nahi reh jaayega.

Isliye teacher ka solution: **Training data ko hi aage do parts mein tod do** — **Train aur Validation**.

### 4. Train aur Validation — Cross Validation Ka Idea

Diagram mein teacher ne Training box ko do parts mein todke **TRAIN** aur **VALIDATION** banaya hai, aur us poore process ko red bracket ke saath "**Cross Validation**" naam diya hai.



(Yahan Train:Validation ka 80:20 ratio sirf samjhane ke liye liya hai — koi fixed rule nahi hai.)

- **Train part** — model is par seekhta hai.
- **Validation part** — is par hum check karte hain ki model ne jo seekha woh kitna sahi hai, aur kaunsi hyperparameter value best hai.
- **Test part** — bilkul alag, sabse last mein sirf ek baar final performance dekhne ke liye.

Analogy: Train = chapters padhna, **Validation** = **mock test / practice paper**, Test = final exam. Mock test ke results dekhkar hum apni taiyari (hyperparameters) sudhaar sakte hain, par final exam ko haath nahi lagate.

## 5. random\_state Ki Problem

Teacher ne Validation box ke saath red colour mein **random\_state** likha hai aur neeche teen accuracies likhi hain: **85%, 92%, 78%**. Yeh ek bahut important point hai.

### random\_state Kya Hota Hai?

Jab hum `train_test_split` se data todte hain, toh records *random* tareeke se alag kiye jaate hain. **random\_state** ek number (seed) hota hai jo decide karta hai ki yeh randomness kaise chalega:

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

- Same random\_state (jaise 42) → **hamesha same split**, same result.
- Different random\_state (jaise 1, 7, 99) → **alag split** → alag Train/Validation records.

### Problem Kya Hai?

Model wahi hai, data wahi hai — bas split alag hai. Par teacher ke example mein alag-alag random\_state par accuracy alag aayi:

|         |       |     |       |
|---------|-------|-----|-------|
| Split A | Train | Val | → 85% |
| Split B | Train | Val | → 92% |
| Split C | Train | Val | → 78% |

■ Train   ■ Validation

Ab sawaal yeh hai — **model ki asli accuracy kaunsi hai: 85%, 92% ya 78%?** Ho sakta hai Split B mein aasaan records validation mein aa gaye (lucky split) aur Split C mein mushkil records aa gaye (unlucky split). Sirf ek split ki accuracy par bharosa karna **risky** hai.

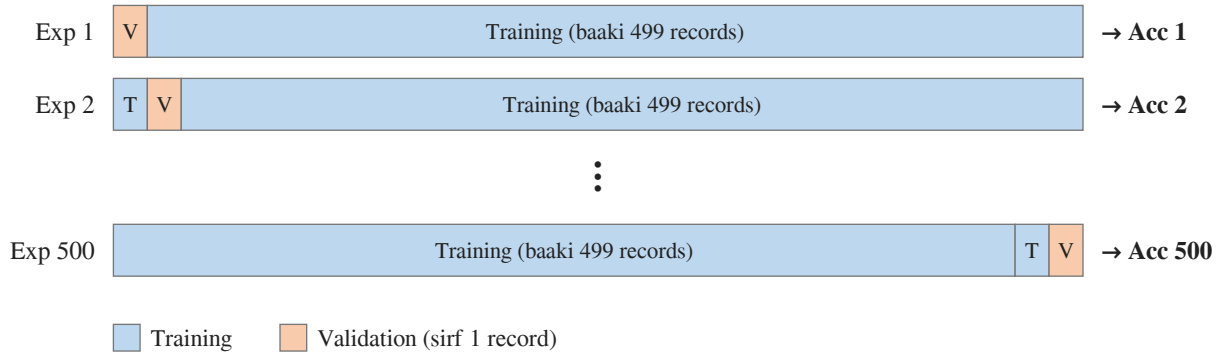
**Yaad Rakhein:** Solution = **Cross Validation**. Alag-alag splits par model ko **multiple baar** train aur validate karo, aur sabhi accuracies ka **average** lo. Isse kisi ek lucky/unlucky split ka asar khatam ho jaata hai. Ab dekhte hain yeh kaam karne ke alag-alag tareeke (types).

## 6. Type 1 — Leave One Out Cross Validation (LOOCV)

**Leave One Out** ka matlab hai — har experiment mein **sirf 1 record ko 'leave out'** (alag) karke usse Validation banana, aur baaki **saare records** par training karna.

## Teacher ka Example

Teacher ne yahan Training data mein **500 records** liye hain. Ab experiments aise challenge:



- **Exp 1:** Pehla record Validation, baaki 499 Training → **Acc 1** milti hai.
- **Exp 2:** Doosra record Validation, baaki 499 Training → **Acc 2** milti hai.
- **Exp 3, Exp 4, ...** aise hi har record ki baari aati hai, ek-ek karke.
- **Exp 500:** Aakhri (500th) record Validation, baaki Training → **Acc 500** milti hai.

Yani **jitne records, utne experiments** (n records = n experiments). Teacher ne in saari accuracies ko ek yellow bracket mein jodkar dikhaya hai — matlab final performance = **saari 500 accuracies ka average**.

**Extra Info:** Classification mein har experiment ka validation set sirf 1 record hota hai, isliye us ek experiment ki accuracy ya toh 100% (sahi predict kiya) ya 0% (galat predict kiya) hoti hai. Sabka average lene par hi meaningful overall accuracy milti hai.

## Fayde (Advantages)

- Almost **saara data training mein use** hota hai (sirf 1 record bahar) — chhote datasets ke liye faydemand.
- **Koi randomness nahi** — random\_state wali problem nahi, har baar same result aata hai.

## Nuksaan (Disadvantages) — Teacher Ke Notes Ke Mutabik

### Disadvantage 1: Training Model Ki Complexity Bahut Badh Jaati Hai

Teacher ne "**500 Records ↑↑ Complexity of Training Model**" likha hai. Kyunki 500 records hain toh model ko **500 baar train** karna padega. Maan lo ek baar training mein 1 second lagta hai:

- 500 records → 500 seconds (lagbhag 8 minute) — theek hai.
- 1,00,000 records → 1,00,000 seconds (lagbhag **27 ghante!**) — bahut zyada costly.

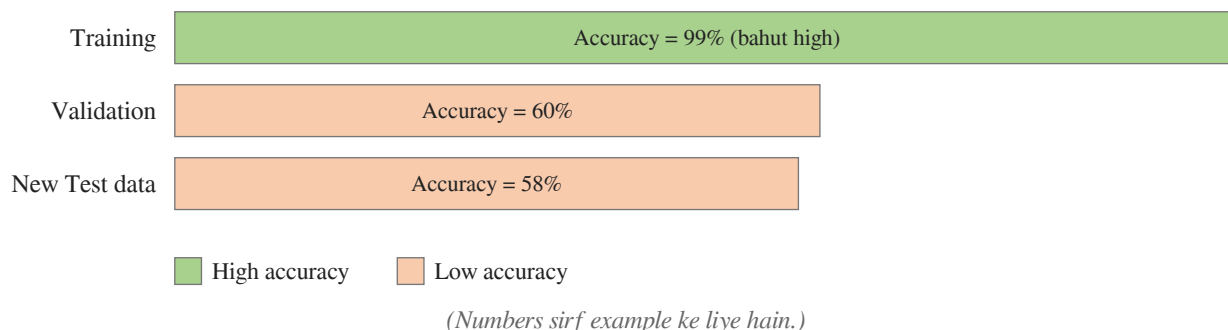
Isliye bade datasets par LOOCV practical nahi hai.

### Disadvantage 2: Overfitting

Teacher ne neeche "**① Overfitting**" likha hai aur ek flow diya hai:

**Training Accuracy ↑↑ → Validation Accuracy ↓↓ → New Test Data par Accuracy ↓↓**

**Overfitting** tab hoti hai jab model training data ko sirf *ratta maar leta hai* (memorize kar leta hai) bajaaye general pattern seekhne ke. Aisa model training data par to kamaal karta hai, par naye data par fail ho jaata hai — bilkul us student ki tarah jisne pichle saal ke sawaal ratta maar liye par concept samjha hi nahi, toh naye paper mein atak jaata hai.



**Yaad Rakhein:** Overfitting ki **pehchaan** yahi hai — Training accuracy aur Validation accuracy ke beech **bada gap**. Cross Validation ka ek bada faayda yahi hai ki hum Validation accuracy dekhkar overfitting pakad sakte hain, kyunki sirf training accuracy dekhne par model hamesha achha lagta hai.

## 7. Type 2 — Leave P Out Cross Validation (LPOCV)

Yeh LOOCV ka hi **bada (generalized) version** hai. LOOCV mein hum har baar **1** record leave out karte the; yahan hum har baar **p records** leave out karte hain. Teacher ne yellow box mein p ki example values likhi hain: **p = 10, p = 20, p = 30**.

- **p = 10:** Har experiment mein 10 records Validation, baaki Training.
- **p = 20:** Har experiment mein 20 records Validation, baaki Training.
- **p = 30:** Har experiment mein 30 records Validation, baaki Training.

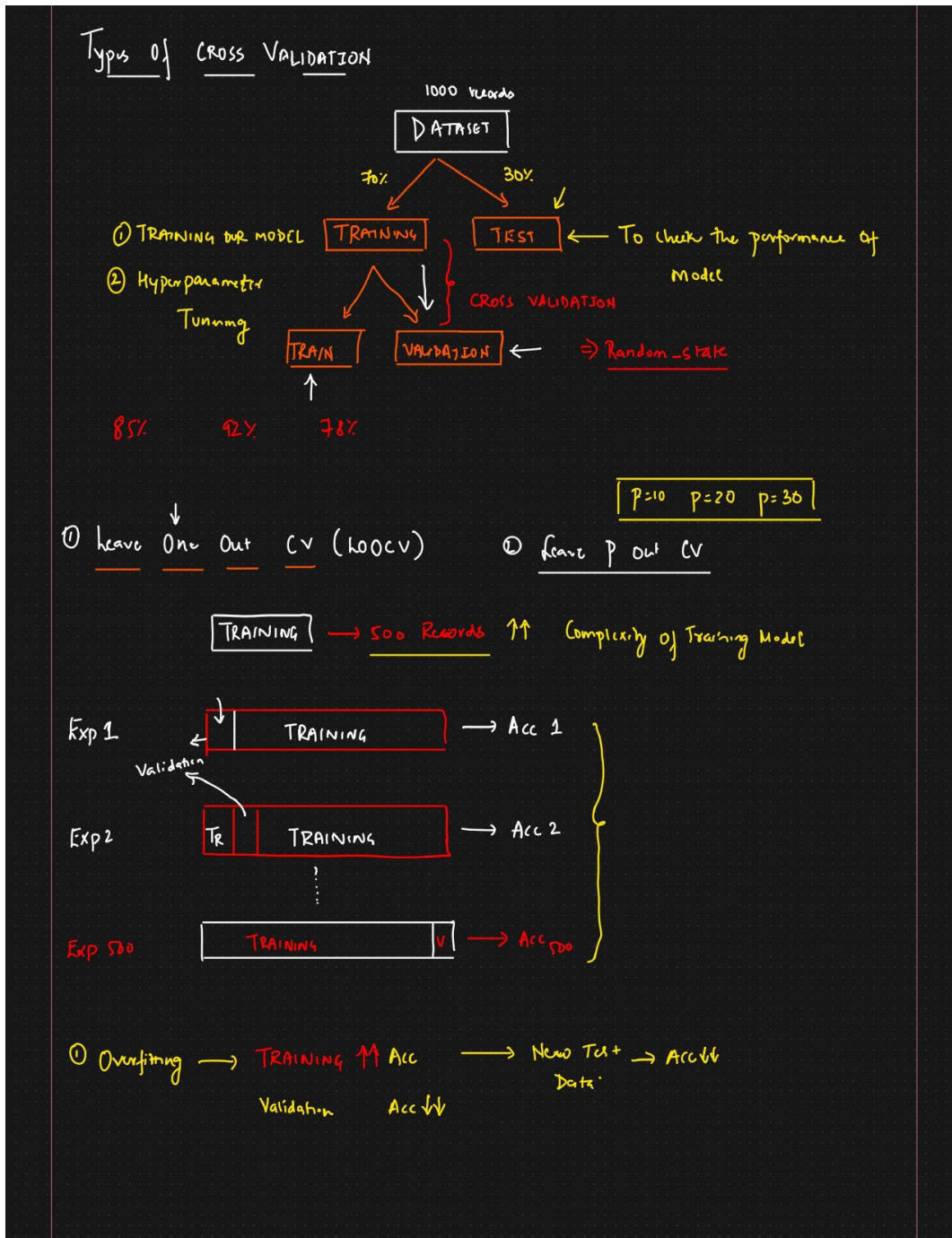
Process bilkul LOOCV jaisa hi hai: har experiment ki accuracy nikaalo, aur aakhir mein sabka average lo.

### Experiments Ki Sankhya

Yahan records ke **saare possible groups (combinations)** try kiye jaate hain. Agar n records mein se p records chunne hain, toh total experiments = **C(n, p)** ("n choose p").

- p = 1 ke liye  $C(n, 1) = n \rightarrow$  yaani **LOOCV hi Leave-P-Out ka special case (p = 1) hai**.
- n = 500 aur p = 10 ke liye  $C(500, 10) \approx 2.5 \times 10^{20}$  experiments — itne experiments kisi bhi computer ke liye practically namumkin hain!

**Dhyaan Dein:** Isi wajah se Leave P Out CV LOOCV se bhi zyada costly hai aur real projects mein shayad hi kabhi use hota hai. Practical kaam ke liye K-Fold (agla type) use kiya jaata hai.



Isme dikhaya gaya hai: Dataset ka 70:30 split, Train/Validation split aur random\_state, LOOCV (Exp 1 se Exp 500), Leave P Out CV ( $p = 10, 20, 30$ ) aur Overfitting.

## 8. Type 3 — K-Fold Cross Validation

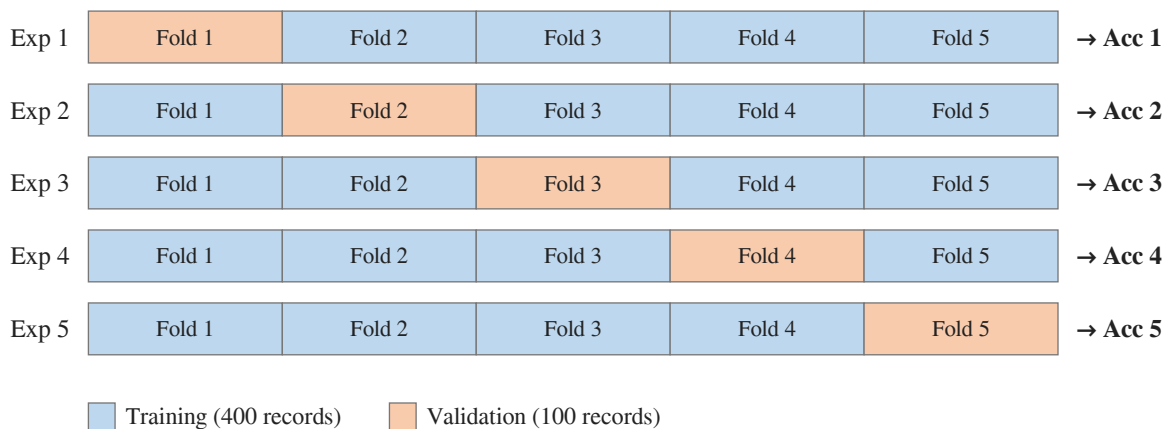
LOOCV mein bahut saare experiments the (500 records = 500 experiments). K-Fold isi problem ko solve karta hai — yahan experiments ki sankhya **K** hoti hai, jo hum khud decide karte hain.

## Teacher Ka Example: $K = 5$ , $n = 500$

Teacher ne example mein  $K = 5$  aur  $n = 500$  records liye hain, aur problem **Binary Classification** hai (yaani output sirf **1** ya **0** hai — jaise sentiment positive/negative).

### Step-by-step process:

- **Step 1:**  $K$  decide karo  $\rightarrow K = 5$ .
- **Step 2:** Data ko  $K$  **equal parts (folds)** mein baanto  $\rightarrow 500 / 5 = 100$  records per fold. Teacher ne isse "Test size =  $500/5 = 100$ " likha hai.
- **Step 3:** Har experiment mein **1 fold Validation** banta hai, aur baaki  $K-1 = 4$  folds (**400 records**) **Training**.
- **Step 4:** Har experiment ki accuracy note karo (Acc 1 ... Acc 5).
- **Step 5:** Sabka **average** lo — yahi model ki final CV accuracy hai (teacher ne "Avg Accuracies" likha hai).



**Average Accuracy** =  $(\text{Acc 1} + \text{Acc 2} + \text{Acc 3} + \text{Acc 4} + \text{Acc 5}) / 5$

Example (sirf samjhane ke liye): agar accuracies 88%, 90%, 86%, 91%, 89% aayi, toh Average =  $(88 + 90 + 86 + 91 + 89) / 5 = 444 / 5 = 88.8\%$ .

**Note:** Teacher ne "Test size" likha hai, lekin yahan iska matlab har fold ka **Validation size** hai (100 records). Asli **Test set** (jo shuru mein 30% alag kiya tha) is process se bilkul alag rehta hai — usse sirf aakhir mein use karte hain.

## K-Fold Ke Fayde

- Har record **exactly 1 baar Validation** mein aata hai aur  $K-1$  baar **Training** mein — data ka poora use hota hai.
- LOOCV se **bahut kam experiments** (500 ki jagah sirf 5) — fast aur practical.
- Ek split ki lucky/unlucky problem (random\_state wali) kaafi had tak khatam ho jaati hai.

**Extra Info:**  $K = n$  rakh do (yaani  $K = 500$  jab  $n = 500$ ), toh K-Fold **LOOCV ban jaata hai**. Practical mein aam taur par  $K = 5$  ya  $K = 10$  use hota hai.

## Python Mein (sklearn):

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)    # cv = K
print(scores.mean())                          # average accuracy
```

## K-Fold Ki Kamzori

Teacher ne example mein "*1 or 0's — Binary Classification*" likha tha. Ab socho ki humare 500 records mein 1's aur 0's barabar nahi hain. K-Fold data ko **random** tareeke se folds mein baantta hai, toh ho sakta hai kisi fold mein zyadatar 1's aa jayein aur kisi mein zyadatar 0's. Aisa hone par us fold ki accuracy bharosemand nahi rehti. Isi problem ko solve karta hai agla type.

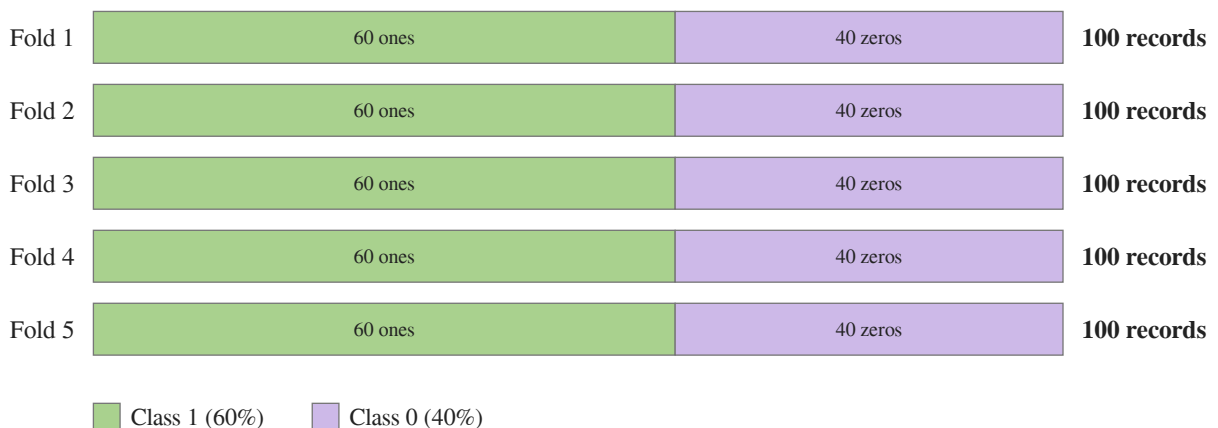
## 9. Type 4 — Stratified K-Fold Cross Validation

**Stratified K-Fold** bhi K-Fold ki tarah hi kaam karta hai, bas ek **extra condition** ke saath: **har fold mein classes (1's aur 0's) ka ratio wahi rehta hai jo poore dataset mein hai.**

### Teacher Ka Example

Teacher ne example mein dataset mein **60 ones (1's)** aur **40 zeros (0's)** liye hain — yaani ratio **60 : 40**.  $K = 5$  aur  $n = 500$  hai, toh har fold mein 100 records honge ("Test size =  $500/5 = 100 \rightarrow$  Validation Data"). Stratified K-Fold ensure karta hai ki:

- Poore dataset mein 500 records = **300 ones + 200 zeros** (60:40).
- Har **Validation fold** (100 records) mein bhi **60 ones + 40 zeros** hongi.
- Har **Training part** (400 records) mein bhi 60:40 ratio hi rahega (240 ones + 160 zeros).



Experiments ka process K-Fold jaisa hi hai: Exp 1 mein Fold 1 Validation aur baaki Training, Exp 2 mein Fold 2 Validation aur baaki Training, ... aur aakhir mein saari accuracies ka average.

## Yeh Zaroori Kyun Hai?

Maan lo ek **imbalanced dataset** hai — 90% records ek class ke aur sirf 10% doosri class ke (jaise fraud detection). Normal K-Fold mein kisi fold mein ho sakta hai ki doosri class ke records bilkul aaye hi nahi. Us fold par model ko kabhi 'test' hi nahi kiya gaya ki woh us class ko pehchaan pata hai ya nahi. Stratified K-Fold har fold mein dono classes ko sahi proportion mein rakhta hai, isliye accuracy zyada **reliable** hoti hai.

**Yaad Rakhein: Classification problems** (khaaskar imbalanced data) mein Stratified K-Fold ko prefer karo. Yeh class labels (1/0) par depend karta hai, isliye continuous output wale Regression problems mein direct use nahi hota.

## 10. Type 5 — Time Series Cross Validation

Ab tak ke saare types mein data ko **random** tareeke se baanta gaya tha. Lekin kuch data mein **time (samay) ka order** bahut important hota hai — wahan random splitting galat hai.

### Teacher Ka Example: Product Sentiment Analysis

Teacher ne example diya hai **Product Sentiment Analysis** ka — yaani kisi product ke **Reviews** padhkar batana ki customer ka sentiment positive hai ya negative. Yeh reviews **time ke saath** aate hain — jaise **JAN** se **DEC** tak, Day 1, Day 2, Day 3, Day 4 ... Day N.

Teacher ke diagram mein data ko aise baanta gaya hai:

- **Training:** Day 1, Day 2, Day 3, Day 4 (pehle ke din — *past*).
- **Validation:** Day 5 se Day N tak (baad ke din — *future*).



## Random Split Yahan Galat Kyun Hai?

Agar hum K-Fold ki tarah random split karein, toh ho sakta hai **December ke reviews Training mein** aa jayein aur **January ke reviews Validation mein**. Yaani model ne **future ka data dekhkar past ko predict** kiya — yeh cheating hai! Real life mein humare paas kal ka data aaj nahi hota. Isse **Data Leakage** kehte hain, aur isse accuracy jhooti (zyada) dikhti hai.

**Yaad Rakhein: Time Series CV ka rule:** Training data hamesha **past** ka ho, aur Validation data hamesha **future** ka. Data ke **order ko kabhi shuffle nahi karte**.

## Practical Mein Yeh Kaise Chalta Hai (Extra Info)

Teacher ne ek hi split dikhaya hai, par practical mein aksar **kai splits** banaye jaate hain jisme Training window aage badhti jaati hai (isse *expanding window* kehte hain), aur sabhi ki accuracy ka average liya jaata hai:

|         |       |       |       |       |       |       |       |       |         |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|---------|
| Split 1 | Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7 | Day 8 | → Acc 1 |
| Split 2 | Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7 | Day 8 | → Acc 2 |
| Split 3 | Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7 | Day 8 | → Acc 3 |

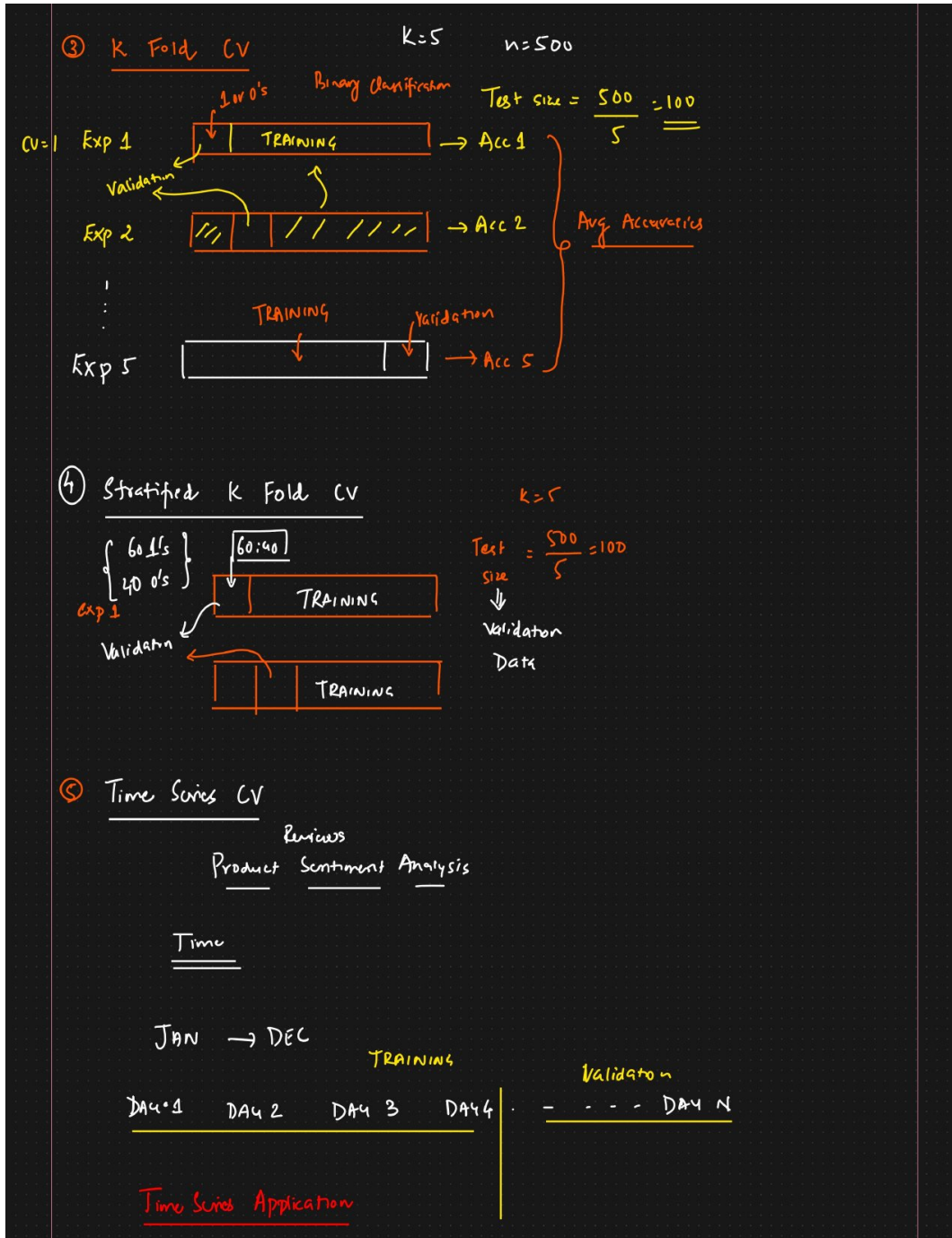
 Training (past)    Validation (next day)    Abhi use nahi hua

Python mein iske liye sklearn ka `TimeSeriesSplit` use hota hai.

## Time Series Application

Teacher ne neeche red mein "**Time Series Application**" likha hai — matlab jahan bhi data time ke saath juda ho wahan yeh CV lagta hai. Jaise:

- Product reviews ka sentiment (jaise upar example mein)
- Stock price prediction
- Weather / temperature forecasting
- Sales aur demand forecasting



Isme dikhaya gaya hai: K-Fold CV ( $K=5$ ,  $n=500$ ), Stratified K-Fold CV (60:40 ratio) aur Time Series CV (Product Sentiment Analysis, Day 1 se Day N).

## 11. Sabhi Types Ki Comparison Table

| Type                     | Validation mein kitna data?     | Kitne experiments?      | Kab use karein?                             | Main dikkat                                          |
|--------------------------|---------------------------------|-------------------------|---------------------------------------------|------------------------------------------------------|
| <b>LOOCV</b>             | Sirf 1 record                   | n (jaise 500)           | Bahut chhota dataset                        | Bahut slow; overfitting; unstable individual results |
| <b>Leave P Out</b>       | p records (10, 20, 30...)       | $C(n, p)$ — bahut zyada | Almost kabhi nahi (theory)                  | LOOCV se bhi zyada costly                            |
| <b>K-Fold</b>            | n / K (500/5 = 100)             | K (jaise 5)             | General purpose, sabse common               | Folds mein class imbalance ho sakta hai              |
| <b>Stratified K-Fold</b> | n / K, class ratio same (60:40) | K (jaise 5)             | Classification, khaaskar imbalanced data    | Regression mein direct use nahi                      |
| <b>Time Series CV</b>    | Future ke din/data              | Splits ke hisaab se     | Time-ordered data (reviews, stock, weather) | Shuffle nahi kar sakte; shuru mein training data kam |

## 12. Kaunsa CV Kab Use Karein? (Quick Guide)

- **Data bahut chhota hai** aur har record valuable hai → LOOCV (ya bada K wala K-Fold).
- **Normal problem, achha size ka data** → K-Fold (K = 5 ya 10).
- **Classification hai aur classes imbalanced hain** (jaise 60:40, 90:10) → Stratified K-Fold.
- **Data time ke order mein hai** (reviews, stock, weather) → Time Series CV.
- **Leave P Out** → concept samajhne ke liye achha hai, par real projects mein use nahi hota.

## 13. Poori Process Ka Summary

- **Step 1:** Dataset lo (jaise 1000 records).
- **Step 2:** Dataset ko Training (70%) aur Test (30%) mein baanto. Test ko abhi haath mat lagao.
- **Step 3:** Training data ka use (1) model train karne aur (2) hyperparameter tuning ke liye hota hai — isliye Training ko aage Train aur Validation mein todo. Yahi Cross Validation hai.
- **Step 4:** Sirf ek random\_state ke split par bharosa mat karo (kabhi 85%, kabhi 92%, kabhi 78%) — Cross Validation ke type se multiple experiments chalao.
- **Step 5:** Apne data ke hisaab se sahi type chuno: LOOCV / Leave P Out / K-Fold / Stratified K-Fold / Time Series CV.
- **Step 6:** Har experiment ki accuracy nikaalo aur unka **average** lo — yeh model ki reliable performance hai.
- **Step 7:** Best hyperparameters chunkar model ko final train karo, aur sabse aakhir mein **Test data** par ek baar check karo.

**Yaad Rakhein: In short:** Cross Validation ka matlab hai — ek hi split par bharosa karne ki jagah data ko alag-alag tareeke se baar-baar Train/Validation mein todke model ko test karna aur average lena, taaki model ki performance **reliable** ho aur **overfitting** pakdi ja sake.

## 14. Important Terms — Chhota Sa Glossary

|                              |                                                                                                       |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Training Data</b>         | Jis data par model seekhta hai.                                                                       |
| <b>Validation Data</b>       | Training data ka woh hissa jis par hum model ko check karte hain aur hyperparameters tune karte hain. |
| <b>Test Data</b>             | Alag rakha hua data, sirf aakhir mein final performance dekhne ke liye.                               |
| <b>Hyperparameter</b>        | Woh setting jo hum khud set karte hain (jaise K in KNN, max_depth, learning rate).                    |
| <b>Fold</b>                  | K-Fold mein data ka ek equal hissa (jaise 500 records, $K = 5 \rightarrow$ har fold mein 100).        |
| <b>random_state</b>          | Ek seed number jo random split ko fix karta hai; alag number = alag split.                            |
| <b>Overfitting</b>           | Jab model training data par bahut achha par naye data par kharab perform kare.                        |
| <b>Binary Classification</b> | Jahan output sirf do classes mein se ek ho (1 ya 0).                                                  |
| <b>Stratified</b>            | Har fold mein classes ka ratio poore dataset jaisa hi rakhna.                                         |
| <b>Data Leakage</b>          | Jab future ya test ki information galti se training mein aa jaaye, aur accuracy jhooti dikhe.         |